Spinlock vs Mutex - Lock Performance Comparison

Theory, Edge Cases & Code Examples

- ✓ Mutex (Blocking Lock)
- ✓ Spin Lock (Busy-Wait Lock)
- ✓ When to Use Which?
- ✓ Priority Inversion
- ✓ Starvation

→ *Swipe to learn* →

Spinlock vs Mutex - Lock Performance Comparison

Theory

📌 1. Mutex (Blocking Lock)

Behavior: Thread sleeps while waiting.

```
std::mutex mtx;

mtx.lock();
// ... critical section ...
mtx.unlock();
```

When lock contended:

1. Thread tries to acquire
2. Fails → **yields to OS scheduler**
3. OS puts thread to sleep
4. Woken when lock available

Cost: Context switch (~1-10 μs)

Spinlock vs Mutex - Lock Performance Comparison

Theory

📌 2. Spin Lock (Busy-Wait Lock)

Behavior: Thread loops (spins) while waiting.

```
std::atomic_flag lock = ATOMIC_FLAG_INIT;

while (lock.test_and_set(std::memory_order_acquire)) {
    // Busy-wait (spin)
}

// ... critical section ...

lock.clear(std::memory_order_release);
```

When lock contended:

1. Thread tries to acquire
2. Fails → **spins in loop**
3. Wastes CPU cycles
4. Acquires immediately when available

Cost: CPU time (but no context switch)

Spinlock vs Mutex - Lock Performance Comparison

Theory

📌 3. When to Use Which?

Use mutex:

- Critical section is long (> 1 μs)
- Many threads contending
- Want to yield CPU to other threads

Use spinlock:

- Critical section is very short (< 100 ns)
- Low contention
- Real-time requirements (avoid context switch)

Complete Implementations

Spinlock vs Mutex - Lock Performance Comparison

Edge Cases

📌 1. Priority Inversion

Problem: High-priority thread spins while low-priority thread holds lock.

Solution: Use mutex (supports priority inheritance on some platforms).

Spinlock vs Mutex - Lock Performance Comparison

Edge Cases

📌 2. Starvation

Problem: Unlucky thread never acquires lock (always preempted).

Solution: Backoff or yield.

Spinlock vs Mutex - Lock Performance Comparison

Code Examples

📌 1. Benchmark Spin Lock vs Mutex (1/3)

This example demonstrates **a direct performance comparison between spin locks and mutexes** under concurrent workload, revealing when each synchronization mechanism is optimal. The benchmark creates multiple threads that repeatedly acquire a lock, increment a counter, and release the lock - simulating a very short critical section typical in high-performance systems.

What this code does:

- Uses a templated `benchmark()` function that works with any lock type (SpinLock or std::mutex)
- Creates 4 worker threads that each perform 100,000 lock-acquire-increment-release cycles
- Measures total execution time using high-resolution clock
- The critical section is minimal (just `++counter`), taking only ~1-2 nanoseconds
- Joins all threads before stopping the timer to measure total parallel execution time

Key concepts demonstrated:

- **Template-based polymorphism** for lock abstraction (works with any type having `lock()` / `unlock()`)
- **Contention measurement** - 4 threads competing for the same lock creates realistic contention
- **Short critical section advantage** - the ~1ns increment operation is where spinlocks excel
- **Context switch overhead** - mutex threads sleep/wake on contention, adding ~1-10μs per cycle
- **Busy-wait efficiency** - spinlock threads immediately acquire when available (no OS scheduler latency)

Why this matters:

In high-frequency trading systems, game engines, and real-time systems, **every microsecond counts**. When critical sections are shorter than a context switch (< 1μs), spinlocks eliminate OS scheduling overhead. However, the benchmark also reveals the tradeoff: spinlocks waste CPU cycles spinning, so they only win when contention is low and critical sections are extremely short.

Performance implications:

- Spinlock wins here because 100,000 × (1-2ns critical section + lock overhead) << context switch cost
- With longer critical sections (>1μs), mutex would win by allowing other work during sleep
- With more threads (>CPU cores), spinlocks degrade as threads spin instead of yielding
- Typical output shows 35-40% speedup for spinlock in this short-critical-section scenario

Real-world applications:

- **Network packet processing**: Incrementing packet counters (critical section ~2ns)
- **Memory allocators**: Updating free lists in lock-free-adjacent designs
- **Game physics**: Collision detection result aggregation
- **High-frequency trading**: Updating order book snapshots

Typical output:

Spin lock wins for short critical sections.

Spinlock vs Mutex - Lock Performance Comparison

Code Examples

📌 1. Benchmark Spin Lock vs Mutex (2/3)

```
#include <iostream>
#include <chrono>
#include <vector>
#include <thread>

template<typename Lock>
void benchmark(const std::string& name, int num_threads, int iterations) {
    Lock lock;
    int counter = 0;

    auto worker = [&]() {
        for (int i = 0; i < iterations; ++i) {
            lock.lock();
            ++counter;
            lock.unlock();
        }
    };

    auto start = std::chrono::high_resolution_clock::now();

    std::vector<std::thread> threads;
    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back(worker);
    }

    for (auto& t : threads) {
        t.join();
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();

    std::cout << name << " (" << num_threads << " threads, " << iterations << " iterations): "
              << ms << " ms\n";
}

int main() {
    benchmark<SpinLock>("Spin Lock", 4, 100000);
    benchmark<std::mutex>("Mutex", 4, 100000);

    return 0;
}
```

Spinlock vs Mutex - Lock Performance Comparison

Code Examples

📌 1. Benchmark Spin Lock vs Mutex (3/3)

```
['Spin Lock (4 threads, 100000 iterations): 42 ms\nMutex (4 threads, 100000 iterations): 65 ms']
```

What You Learned

Key Takeaways

- ✓ Mutex (Blocking Lock)
- ✓ Spin Lock (Busy-Wait Lock)
- ✓ When to Use Which?
- ✓ Priority Inversion

→ Swipe to learn →